

Task 1:

Wap to create a class Student.. With 3 methods
registrationDetails(), marksCalc(), feesCalc().

```
class Student {  
  
    // Method 1: Registration Details  
    void registrationDetails() { 1 usage  
        System.out.println("Registration Details method called.");  
    }  
  
    // Method 2: Marks Calculation  
    void marksCalc() { 1 usage  
        System.out.println("Marks Calculation method called.");  
    }  
  
    // Method 3: Fees Calculation  
    void feesCalc() { 1 usage  
        System.out.println("Fees Calculation method called.");  
    }  
  
    public static void main(String[] args) {  
        Student s1 = new Student();  
  
        s1.registrationDetails();  
        s1.marksCalc();  
        s1.feesCalc();  
    }  
}
```

```
C:\Program Files\Java\jdk-17\bin\java.exe
Registration Details method called.
Marks Calculation method called.
Fees Calculation method called.

Process finished with exit code 0
```

Task2

```
class Employee {
    private String name;
    private String email;
    private double salary;

    // Methods related to employee data

    // Method to generate PDF report
    public void generatePdfReport() {
        // Code to generate PDF report
    }

    // Method to send email
    public void sendEmail() {
        // Code to send email
    }
}
```

In the above example code, the Employee class violates the SRP because it has multiple responsibilities: managing employee data, generating PDF reports, and sending emails. These responsibilities are not cohesive and may change for different reasons.

```
// Employees.java
class Employee { 4 usages
    private String name; 2 usages
    private String email; 2 usages
    private double salary; 2 usages

    // Constructor
    public Employee(String name, String email, double salary) { 1 usage
        this.name = name;
        this.email = email;
        this.salary = salary;
    }

    // Getters
    public String getName() { 1 usage
        return name;
    }

    public String getEmail() { 1 usage
        return email;
    }

    public double getSalary() { no usages
        return salary;
    }
}
```

```

5     }
6 }
7
8 // ReportGenerator.java
9 class ReportGenerator { 2 usages
10 @ public void generatePdfReport(Employee employee) { 1 usage
11     System.out.println("Generating PDF report for " + employee.getName());
12     // Placeholder for actual PDF logic
13 }
14 }
15
16 // EmailService.java
17 class EmailService { 2 usages
18 @ public void sendEmail(Employee employee) { 1 usage
19     System.out.println("Sending email to " + employee.getEmail());
20     // Placeholder for actual email logic
21 }
22 }
23
24 // Main.java
25 ▶ public class Employees {
26 ▶     public static void main(String[] args) {
27         // Create employee object
28         Employee emp = new Employee(name: "Alice", email: "alice@example.com", salary: 50000)

```

```

// Main.java
public class Employees {
    public static void main(String[] args) {
        // Create employee object
        Employee emp = new Employee(name: "Alice", email: "alice@example.com", salary: 50000);

        // Use helper services
        ReportGenerator reportGen = new ReportGenerator();
        EmailService emailSvc = new EmailService();

        // Call methods
        reportGen.generatePdfReport(emp);
        emailSvc.sendEmail(emp);
    }
}

```

```
C:\Program Files\Java\jdk-17\bin\java.  
Generating PDF report for Alice  
Sending email to alice@example.com  
  
Process finished with exit code 0
```

Task 03:

/*

In the below example, the `PaymentProcessor` class violates the OCP because every time a new payment type needs to be supported, the `processPayment` method needs to be modified, which is against the principle of being closed for modification.

*/

```
class PaymentProcessor {  
    public void processPayment(String paymentType, double amount) {  
        if (paymentType.equals("credit card")) {  
            // Process credit card payment  
        } else if (paymentType.equals("paypal")) {  
            // Process PayPal payment  
        } else if (paymentType.equals("bitcoin")) {  
            // Process Bitcoin payment  
        }  
        // ... more payment types  
    }  
}
```

```

// Interface for payment methods
interface PaymentMethod { 7 usages 3 implementations
    void processPayment(double amount); 1 usage 3 implementations
}

// Implementation for Credit Card payment
class CreditCardPayment implements PaymentMethod { 1 usage
    public void processPayment(double amount) { 1 usage
        System.out.println("Processing credit card payment of $" + amount);
    }
}

// Implementation for PayPal payment
class PaypalPayment implements PaymentMethod { 1 usage
    public void processPayment(double amount) { 1 usage
        System.out.println("Processing PayPal payment of $" + amount);
    }
}

// Implementation for Bitcoin payment
class BitcoinPayment implements PaymentMethod { 1 usage
    public void processPayment(double amount) { 1 usage
        System.out.println("Processing Bitcoin payment of $" + amount);
    }
}

// PaymentProcessor class that uses PaymentMethod

```

Activate Wi

```

class BitcoinPayment implements PaymentMethod { 1 usage
    public void processPayment(double amount) { 1 usage
        System.out.println("Processing Bitcoin payment of $" + amount);
    }
}

// PaymentProcessor class that uses PaymentMethod
class PaymentProcessor { 2 usages
    public void processPayment(PaymentMethod paymentMethod, double amount) { 3 us
        paymentMethod.processPayment(amount);
    }
}

// Main class to test payment processing
public class Cod {
    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();

        PaymentMethod creditCard = new CreditCardPayment();
        PaymentMethod paypal = new PaypalPayment();
        PaymentMethod bitcoin = new BitcoinPayment();

        processor.processPayment(creditCard, amount: 150.0);
        processor.processPayment(paypal, amount: 250.0);
        processor.processPayment(bitcoin, amount: 350.0);
    }
}

```

Activate

```
Rename usages
// Main class to test payment processing
public class Cod {
    public static void main(String[] args) {
        PaymentProcessor processor = new PaymentProcessor();

        PaymentMethod creditCard = new CreditCardPayment();
        PaymentMethod paypal = new PaypalPayment();
        PaymentMethod bitcoin = new BitcoinPayment();

        processor.processPayment(creditCard, amount: 150.0);
        processor.processPayment(paypal, amount: 250.0);
        processor.processPayment(bitcoin, amount: 350.0);
    }
}
```

```
Processing credit card payment of $150.0
Processing PayPal payment of $250.0
Processing Bitcoin payment of $350.0

Process finished with exit code 0
```

Task04:

Sequence diagram:

@startuml

Prasunamba -> Atlas_Batch1: Congratulations all

Atlas_Batch1 --> Prasunamba: Thank you Meher

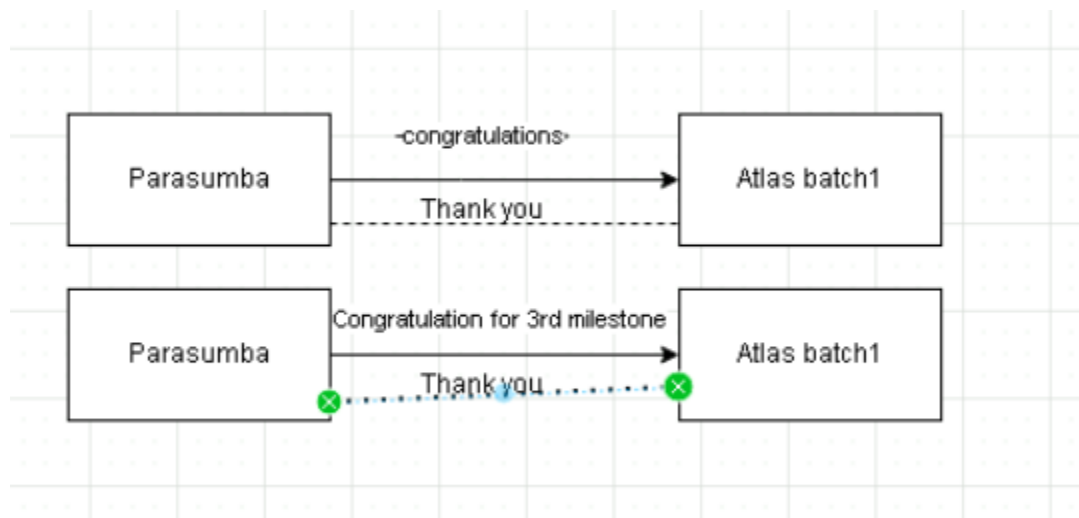
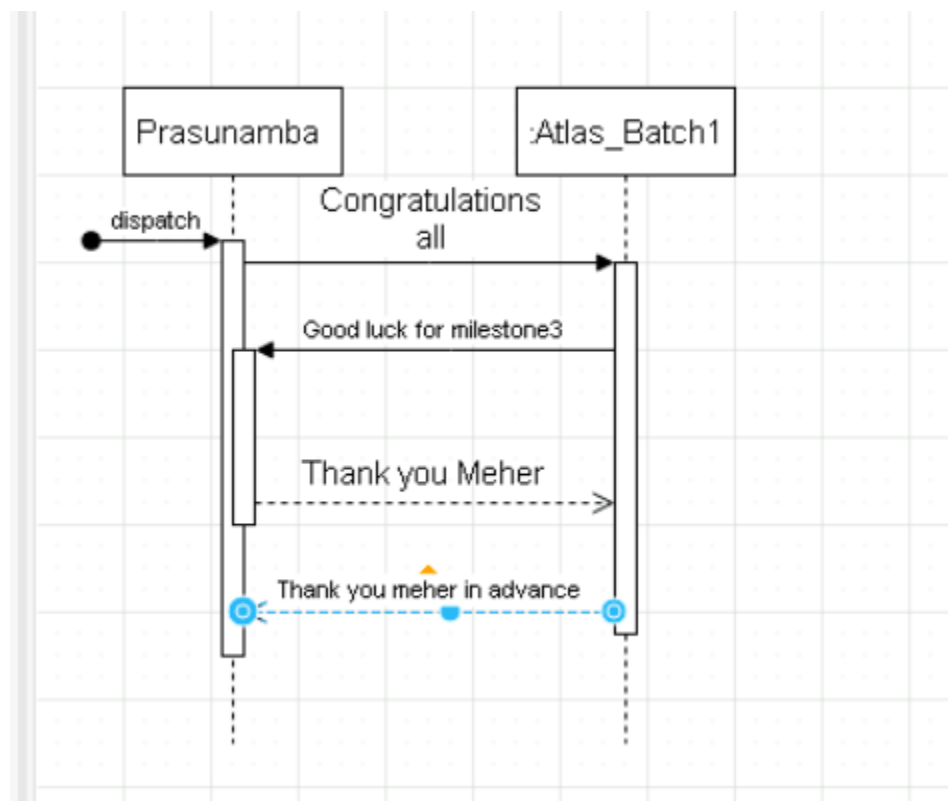
Prasunamba -> Atlas_Batch1: Good luck for your Module 3

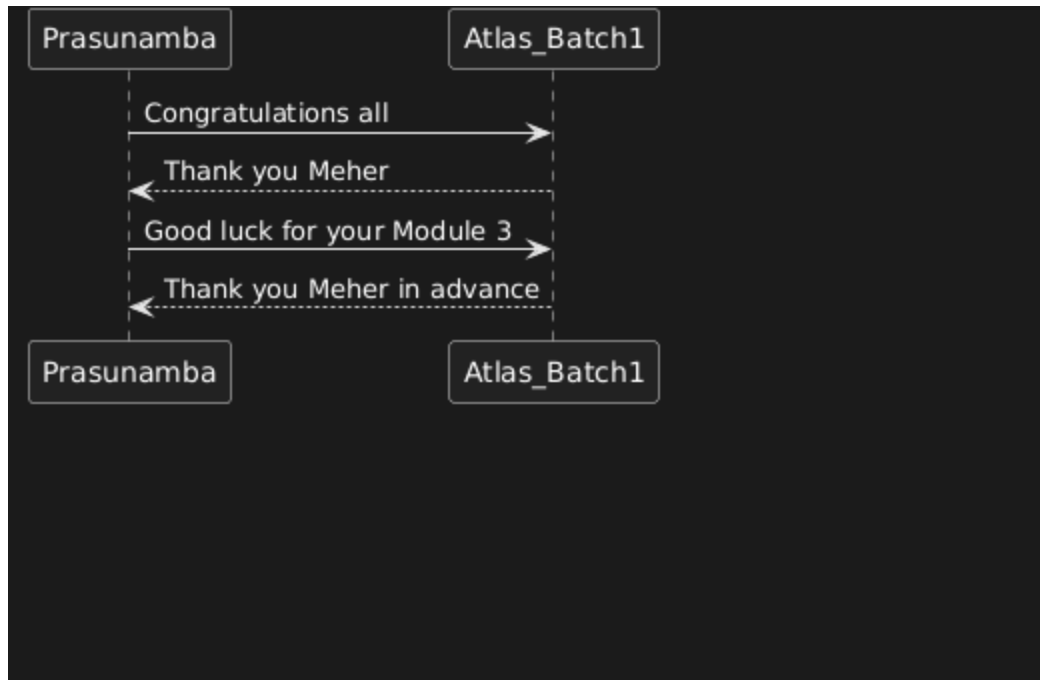
Atlas_Batch1 --> Prasunamba: Thank you Meher in advance

or

Prasunamba <-- Atlas_Batch1: Thank you Meher in advance

@enduml





Task 5:

@startuml

pax "A" as A

pax "B" as B

A --> B: Message/action

A ..> B: reply

@enduml

